

Lesson 2: Branching, Merging & Handling Conflicts

In Lesson 1, you learned how to track changes locally on a single line of history. In this lesson, we cover one of Git's most powerful features: Branching. Branching allows team members to work on new features or bug fixes in complete isolation without breaking the main codebase.

1. Understanding Git Branches

Think of a branch as an independent timeline of your project. Every repository starts with a default branch (usually named main or master).

When you create a new branch:

- You take a snapshot of the code at that exact moment.
- Changes made on the new branch do not affect main.
- Once your feature is complete and tested, you merge those changes back into main.

(feature-login) ---> [Commit C] ---> [Commit D]

\

\ (Merge)

[Commit A] -----> [Commit B] -----> [Commit E] (main)

2. Essential Branching Commands

Open your terminal in your project directory to practice creating and navigating branches:

Bash

List all local branches (the * indicates your current branch)

`git branch`

Create a new branch named 'feature-login'

`git branch feature-login`

Switch to the new branch

`git switch feature-login`

Shortcut: Create and switch to a new branch in one command

`git switch -c feature-dashboard`

3. The Feature Branch Workflow

Here is the standard workflow developers follow every day:

1. Switch to main and pull latest changes:

Bash

```
git switch main
```

2. Create a fresh feature branch:

Bash

```
git switch -c add-user-profile
```

3. Make your changes, stage, and commit:

Bash

```
# Edit your files...
```

```
git add .
```

```
git commit -m "Add user profile layout and styles"
```

4. Switch back to main to merge:

Bash

```
git switch main
```

```
git merge add-user-profile
```

5. Delete the feature branch once merged:

Bash

```
git branch -d add-user-profile
```

4. Understanding & Resolving Merge Conflicts

A merge conflict happens when two different branches change the same line of code in different ways, and Git doesn't know which version is correct.

When a conflict occurs during git merge, Git stops and modifies the affected file with conflict markers:

Plaintext

```
<<<<<<< HEAD
```

```
console.log("Welcome to the Admin Dashboard!");
```

```
=====
```

```
console.log("Welcome back, User!");
```

>>>>>> feature-dashboard

How to Fix a Conflict:

1. Open the conflicted file in your editor.
2. Decide which code to keep (or combine both) and remove the <<<<<<, =====, and >>>>>> lines.
3. Save the file.
4. Stage and commit the resolved file:

Bash

git add .

git commit -m "Fix merge conflict in dashboard logic"

Practical Exercise

Try this hands-on scenario to practice branching and resolving a conflict:

1. On main, edit notes.txt to add: Version 1: Base settings. Commit this change.
2. Create and switch to a new branch called update-notes.
3. In notes.txt, change line 1 to: Version 1: Updated on branch. Commit this change.
4. Switch back to main, and change line 1 of notes.txt to: Version 1: Updated on main. Commit this change.
5. Run git merge update-notes. Git will trigger a merge conflict.
6. Open notes.txt, pick one version, remove the conflict markers, save, and complete the merge with git add . and git commit -m "Resolve notes conflict".